CYBERSHIELD

A Project Report

Submitted in partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

Submitted By

BOBBY VIJAY SINGH

Under the Guidance of

Mrs. M. D. Hindurao

DEPARTMENT OF COMPUTER SCIENCE

Janseva Shikshan Mandal's

SHANTARAMBHAU GHOLAP ARTS, SCIENCE AND GOTIRAMBHAU PAWAR COMMERCE COLLEGE SHIVLE

(Affiliated to University of Mumbai)

SHIVALE (MURBAD) – 421401

MAHARASHTRA

2026-27

SHANTARAMBHAU GHOLAP ARTS, SCIENCE AND GOTIRAMBHAU PAWAR COMMERCE COLLEGE SHIVLE

(Affiliated to University of Mumbai)

SHIVLE (MURBAD)-MAHARASHTRA-421401

DEPARTMENT OF COMPUTER SCIENCE



CERTIFICATE

This is to certify that the project entitled, **"CYBERSHIELD"** is the bonafide work of **BOBBY VIJAY SINGH**, submitted in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** from the University of Mumbai, under the guidance of **Mrs. M. D. Hindurao.**

Internal Guide

Internal Examiner **External Examiner**

Date: **Collage Seal** **Principal**

PROJECT PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

1. Name of the Student : **BOBBY VIJAY SINGH**
2. Title of the Project : **CYBERSHIELD**
3. Name of the Guide : **Mrs. M. D. Hindurao**
4. Teaching experience of the Guide : **Excellent**
5. Is this your first submission? **Yes**

Signature of the Student

Date: ……

Signature of the Guide

Date: ……

Signature of the Coordinator

Date: ……

ABSTRACT

CyberShield is a web-based smart threat detection application that lets any person check whether a link, a file, an SMS or email message, a QR code, an IP address or a password is dangerous. With the rapid growth of digital payments, online banking and messaging in India, ordinary users receive many suspicious links and messages every day. Most people have no simple way to find out whether a link is a phishing page, whether a downloaded file contains malware, or whether a message is a KYC, OTP or job scam.

The CyberShield project addresses this problem by combining several real security services behind one simple interface. A link is checked against Google Safe Browsing, more than seventy antivirus engines through VirusTotal, the registration age of its domain through RDAP, and a set of phishing heuristics. A file is identified by its SHA-256 hash, which is computed inside the browser, so that known files are looked up without uploading anything. Messages are analysed by Gemini AI together with a rule engine that recognises common Indian scam patterns in English and Hinglish. QR codes are decoded in the browser and UPI payment requests are explained with a clear warning that nobody needs to scan a QR code to receive money. IP addresses are checked against AbuseIPDB, and passwords are checked against the Have I Been Pwned breach database using k-anonymity so the password itself never leaves the browser.

All signals are combined by a weighted threat score engine into a 0 to 100 score and a verdict of Safe, Suspicious or Dangerous, together with a plain-language explanation, a detailed breakdown of findings and a downloadable PDF report. The application also contains an awareness section describing common scams and a ten-question quiz.

CyberShield is developed using React, Vite, Tailwind CSS and Framer Motion on the front end, Node.js and Express on the back end, and Cloud Firestore for a public scan counter. All API keys stay on the server, requests are rate limited and cached, and every external source can fail without breaking the result. The project demonstrates the practical use of cybersecurity services, secure API design and privacy-preserving techniques in a modern web application.

Keywords: Cybersecurity, Phishing Detection, Threat Score, VirusTotal, Google Safe Browsing, k-Anonymity, QR Code, UPI Fraud, React, Node.js, Express, Gemini AI.

ACKNOWLEDGMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the development of my project “CYBERSHIELD.”

First of all, I am deeply thankful to my project guide, **Mrs. M. D. Hindurao**, for her valuable guidance, continuous support, encouragement, and constructive suggestions throughout the development of this project and preparation of the project report.

I am grateful to the Department of Computer Science and the management of **Shantarambhau Gholap Arts, Science and Gotirambhau Pawar Commerce College, Shivle**, for providing the necessary facilities, resources, and academic support required to complete this project successfully.

I would also like to thank all the teaching and non-teaching staff members of the department for their valuable assistance and encouragement.

Finally, I express my sincere thanks to my friends, classmates, and family members for their constant support, motivation, and encouragement during the completion of this project.

Thank You!

Presented By

BOBBY VIJAY SINGH

DECLARATION

I hereby declare that the project report entitled "**CYBERSHIELD**" is my original work carried out under the guidance of **Mrs. M. D. Hindurao**, Department of Computer Science, Shantarambhau Gholap Arts, Science and Gotirambhau Pawar Commerce College, Shivle, affiliated to the University of Mumbai.

I further declare that this project has been prepared and submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Science (Computer Science).

I also declare that, to the best of my knowledge, this project report has not been submitted previously, either in part or in full, to any other university or institution for the award of any degree or qualification.

Name: **BOBBY VIJAY SINGH**

Signature: _____

Date: _____

INDEX

SR.NO	CONTENTS	PAGE NO
1	Certificate	2
2	Project Proforma for the Approval Project Proposal	3
3	Abstract	4
4	Acknowledgment	5
5	Declaration	6
6	INDEX	7
7	CHAPTER 1 – INTRODUCTION	8-10
8	CHAPTER 2 – LITERATURE REVIEW / RELATED WORK	11-13
9	CHAPTER 3 – METHODOLOGY	14-15
10	CHAPTER 4 – IMPLEMENTATION DETAILS	16-18
11	CHAPTER 5 – SYSTEM DESIGN	19-22
12	CHAPTER 6 – EXPERIMENTAL SETUP & RESULT	23-29
13	CHAPTER 7 – ANALYSIS AND REPORT	30-31
14	CHAPTER 8 – FUTURE ENHANCEMENTS	32
15	CHAPTER 9 – PROGRAM CODE	33-39
16	CHAPTER 10 – CONCLUSION	40
17	REFERENCE	41

CHAPTER 1 INTRODUCTION

1.1 INTRODUCTION

Nowadays, computers, mobile phones and the Internet are used for almost every daily activity. People use websites and applications for communication, banking, shopping, education and government services. Because of this, users constantly receive links, files and messages from unknown sources through SMS, WhatsApp, email and social media.

At the same time, online fraud has increased rapidly. Phishing links copy the look of bank websites, downloaded files may contain malware, and scam messages threaten users with account blocking or tempt them with fake job offers and prizes. Most users cannot easily tell whether a link, file or message is genuine.

CyberShield is a web-based smart threat detection application developed to solve this problem. The system allows a user to check six different types of input: a link, a file, an SMS or email message, a QR code, an IP address and a password. Each input is checked against real security services and the result is shown as a threat score between 0 and 100, a verdict of Safe, Suspicious or Dangerous, and a simple explanation in plain language.

The application uses Google Safe Browsing and VirusTotal for links and files, Gemini AI together with a rule engine for messages, AbuseIPDB for IP addresses, and the Have I Been Pwned breach database for passwords. QR codes are decoded directly in the browser, and UPI payment QR codes are explained with a warning about the common "scan to receive money" scam.

CyberShield is developed using **React, Vite, Tailwind CSS, Framer Motion, Node.js, Express and Cloud Firestore**. The main aim of the project is to provide a free, simple and trustworthy tool that helps ordinary users stay safe online.

1.2 BACKGROUND

The use of digital payments and online services in India has grown very quickly in recent years. UPI, net banking and e-commerce are used by crores of people, including many who are new to technology. Fraudsters take advantage of this by sending fake KYC update links, OTP requests, electricity-bill disconnection warnings, lottery messages and job offers.

Security companies such as Google and VirusTotal maintain large databases of dangerous websites and files, but these services are mostly used by experts and other software. A normal user does not know how to query such services, and checking each source separately is slow and confusing.

To solve these problems, CyberShield combines several security services in one application. The backend queries the services in parallel, combines their answers into one score, and the frontend presents the result in a visual and easy-to-understand way. The project also teaches users about common scams through an awareness page and a quiz.

The project is developed as a practical application of cybersecurity concepts such as threat intelligence, hashing, k-anonymity, API security, rate limiting and secure deployment.

1.3 OBJECTIVE

The main objective of the CyberShield project is to develop a simple and reliable web application that tells a user whether a link, file, message, QR code, IP address or password is dangerous.

The objectives of the project are:

1. To check links against Google Safe Browsing, VirusTotal, domain age and phishing heuristics.
2. To scan files using a browser-side SHA-256 hash and VirusTotal, without storing any file on the server.
3. To detect scam SMS and email messages using Gemini AI and a rule engine for Indian scam patterns.
4. To decode QR codes in the browser and warn users about UPI "receive money" frauds.
5. To check the reputation of an IP address using AbuseIPDB and geolocation data.
6. To check whether a password has appeared in known data breaches using k-anonymity, without sending the password anywhere.
7. To combine all signals into a weighted threat score and a clear verdict.
8. To explain every result in plain language with a detailed breakdown and a downloadable PDF report.
9. To keep all API keys on the server and protect the API with CORS, validation, caching and rate limiting.
10. To increase user awareness of common scams through an awareness page and a quiz.

1.4 PURPOSE, SCOPE AND APPLICABILITY

1.4.1 PURPOSE

The purpose of CyberShield is to give every user, even without technical knowledge, a quick and dependable way to check suspicious items before clicking, opening, scanning or paying.

The system brings together the answers of several professional security services, converts them into one simple score and explains the reasons behind it. It also protects the privacy of the user, because files are identified by hash and passwords never leave the browser.

The project is mainly developed to understand how real-world security APIs, scoring logic and secure web architecture can be combined into one practical application.

1.4.2 SCOPE

The scope of the CyberShield project includes:

- Link scanning with Google Safe Browsing, VirusTotal, RDAP domain age and heuristics
- File scanning with browser-side SHA-256 and VirusTotal hash-first lookup
- SMS and email scam detection with Gemini AI and a rule engine (English and Hinglish)
- QR code scanning by camera, image upload or pasted text, with UPI parsing

- IP address reputation with AbuseIPDB and geolocation
- Password leak checking with k-anonymity and a local strength meter
- Weighted threat score engine with Safe / Suspicious / Dangerous verdicts
- Result page with animated gauge, findings, engine grid and PDF report
- Public live scan counter stored in Cloud Firestore
- Scam awareness page with seven scam types, helplines and a ten-question quiz
- Secure API with CORS restriction, input validation, caching and rate limiting
- Deployment of the client on GitHub Pages and the API on Render

In the future, the system can be extended with user accounts, browser extensions and a mobile application.

1.4.3 APPLICABILITY

CyberShield can be useful for anyone who receives suspicious links, files or messages. The system can be used for:

Personal Use:

Users can check a link or message received on WhatsApp or SMS before opening it, and check whether their passwords have been leaked.

Educational Use:

Students can use the project to understand phishing detection, threat scoring, hashing, k-anonymity, REST APIs and secure deployment.

Small-Scale Use:

Shops, offices and senior citizens' groups can use the awareness page and the scanners to avoid common UPI, KYC and OTP frauds.

Overall, CyberShield provides a simple way to understand and apply important cybersecurity concepts through a modern web application.

CHAPTER 2

LITERATURE REVIEW / RELATED WORK

2.1 INTRODUCTION

Literature review helps to understand the existing methods and systems related to the project. In the field of cybersecurity, many services and techniques are available for identifying malicious websites, malware files, scam messages and leaked passwords.

Large companies maintain threat intelligence databases, antivirus vendors share detections through aggregation platforms, and researchers have developed machine learning methods for detecting phishing. Privacy-preserving methods such as k-anonymity have also been developed for checking sensitive data safely.

This chapter studies the basic concepts and existing methods related to URL reputation, multi-engine file scanning, scam message detection, IP reputation and breach checking. It also discusses the limitations of existing methods and explains the need for a simple system like CyberShield.

2.2 TRADITIONAL METHODS OF CHECKING LINKS

Traditionally, users judge a link by looking at it, by asking a friend, or by opening it carefully. Some users search the website name online or look for a padlock icon in the browser. These methods are unreliable because fraudsters copy the design of real websites, use look-alike domain names such as "sbi-kyc-update" and obtain HTTPS certificates for free.

Modern browsers warn users about some dangerous websites using built-in blocklists, but these warnings only appear after the user has already clicked the link. A tool that can check a link before it is opened gives users an important advantage.

2.3 URL REPUTATION SERVICES

Google Safe Browsing is a service that maintains constantly updated lists of websites that host malware, phishing pages, unwanted software and potentially harmful applications. Applications can query the Safe Browsing Lookup API with a URL and receive the matching threat types.

VirusTotal is an aggregation platform that submits a URL or file to more than seventy antivirus engines and website scanners and returns each engine’s verdict. It also stores historical analyses so that known items can be looked up instantly by their identifier.

Domain registration data from RDAP (Registration Data Access Protocol) shows when a domain was created. Phishing domains are often only a few days old because they are created, used and abandoned quickly.

CyberShield uses all three sources together. Because each source answers a different question, combining them gives a more reliable verdict than any single source.

2.4 MULTI-ENGINE FILE SCANNING AND HASHING

A cryptographic hash such as SHA-256 is a fixed-length fingerprint of a file. Two different files practically never produce the same hash, so a hash can identify a file without revealing its contents.

Services like VirusTotal allow files to be looked up by hash. If the file has already been analysed, the complete result is available instantly. Only unknown files need to be uploaded for a fresh analysis.

CyberShield follows this "hash-first" approach. The hash is calculated inside the user's browser using the Web Crypto API, so the file itself is uploaded only when necessary and is never stored on the project server.

2.5 SCAM MESSAGE DETECTION AND ARTIFICIAL INTELLIGENCE

Scam messages follow recognisable patterns: urgency ("your account will be blocked today"), requests for secret codes ("share the OTP"), rewards ("you have won a lottery") and advance fees ("pay the registration fee"). Early systems used keyword lists to detect such patterns.

Large language models such as Google Gemini can read a message and explain why it is suspicious, identify the scam type and highlight the exact phrases that are red flags. They understand mixed languages such as Hinglish, which keyword lists handle poorly.

CyberShield uses Gemini with a strict JSON output format and combines it with a local rule engine. The rule engine keeps the system working when the AI service is slow or its free quota is exhausted, and it adds red flags the AI may have missed.

2.6 IP REPUTATION AND PASSWORD BREACH CHECKING

AbuseIPDB is a community database in which administrators report IP addresses involved in spam, brute-force attacks and other abuse. The service returns an abuse confidence score, report counts, the ISP and the usage type of an address.

Have I Been Pwned (HIBP) maintains a database of more than a billion passwords that have appeared in data breaches. Its Pwned Passwords API uses **k-anonymity**: the client sends only the first five characters of the SHA-1 hash of the password, receives all hash suffixes that share that prefix, and compares them locally. The service never learns the password or even its full hash.

CyberShield implements both techniques. The password check runs entirely in the browser and never contacts the project's own server.

2.7 LIMITATIONS OF EXISTING SYSTEMS

Existing methods and tools have some limitations:

- Professional services such as VirusTotal and Safe Browsing are designed for developers and are not easy for ordinary users to use.
- Each service answers only one question; a user must check several websites to get the complete picture.
- Browser warnings appear only after a link is clicked.
- Most tools do not explain the result in simple language or suggest what the user should do.
- Few tools understand Indian scam patterns such as KYC, UPI and electricity-bill frauds, or messages written in Hinglish.
- Checking a password on an untrusted website is itself a security risk if the password is sent to a server.

Because of these limitations, there is a need for a simple system that combines different security sources in one application while protecting the user's privacy.

2.8 RESEARCH GAP

From the study of existing methods, it can be observed that strong security services exist but are fragmented and expert-oriented. There is no simple, free tool aimed at everyday users in India that checks links, files, messages, QR codes, IP addresses and passwords in one place and explains the result.

CyberShield attempts to fill this gap. It combines Google Safe Browsing, VirusTotal, RDAP, AbuseIPDB, Have I Been Pwned and Gemini AI behind one interface, converts their answers into a single weighted score, explains the findings in plain language and keeps the user's data private.

The project is mainly focused on providing a simple and trustworthy system that demonstrates practical cybersecurity concepts. It can also be extended in the future with additional sources and features.

CHAPTER 3 METHODOLOGY

3.1 PROBLEM DEFINITION

Nowadays, users receive many links, files and messages from unknown senders. Fraudsters use fake bank pages, malware files, scam messages and payment QR codes to steal money and personal information. Most users have no simple way to check whether such an item is safe before interacting with it.

Professional security services exist, but they are separate, technical and give raw data instead of advice. Checking a password for leaks is also risky if the password has to be sent to a server.

To solve these problems, CyberShield is developed as a web-based threat detection application. The system combines several security services, computes a weighted threat score, explains the result and protects the privacy of the user.

The main problem addressed by the project is to provide a simple system through which users can check links, files, messages, QR codes, IP addresses and passwords and receive a clear, explained verdict.

3.2 SYSTEM ARCHITECTURE

The CyberShield system follows a client-server architecture. It consists of different components that work together to provide the threat detection features to the user.

1. User Interface (Client)

The user interface is developed using React with Vite, Tailwind CSS and Framer Motion. It provides pages for the home screen, the scanner with six tabs, the result page, the awareness page and the quiz. The interface uses a dark "cyber command center" design with a radar scan animation and an animated threat gauge.

2. Application Layer (API)

The application logic is developed using Node.js and Express. The API exposes routes for scanning links, files, messages and IP addresses. It validates input, applies rate limiting, queries the external services in parallel, computes the threat score and returns a JSON report.

3. Scanner Modules

Each scanner is implemented as a separate module: Link Scanner, File Scanner, SMS/Email Scam Checker, IP Address Checker, and, on the client side, the QR Code Scanner and the Password Leak Checker.

4. Threat Score Engine

The threat score engine contains all scoring weights in one configuration object. It converts the signals from each source into points, adds them up with a cap of 100 and maps the total to a verdict: 0–30 Safe, 31–60 Suspicious, 61–100 Dangerous.

5. External Security Services

Google Safe Browsing, VirusTotal, AbuseIPDB, RDAP, ip-api and Gemini AI are called from the server. API keys are stored only on the server in environment variables. Have I Been Pwned is called directly from the browser using k-anonymity.

6. Cache and Rate Limiting

An in-memory cache stores reports for one hour so that repeated scans of the same link or file do not use the VirusTotal quota again. A rate limiter allows 20 scans per 10 minutes per IP address.

7. Database Layer

Cloud Firestore stores a single public counter of the total number of scans, which is shown live on the home page. No personal data is stored.

3.3 WORKING OF THE SYSTEM

The basic working process of CyberShield is as follows:

1. The user opens the application and selects a scanner tab (Link, File, SMS/Email, QR Code, IP Address or Password).
2. The user enters a link, chooses a file, pastes a message, scans a QR code or types a password.
3. The client validates the input. For a file it computes the SHA-256 hash; for a password it computes the SHA-1 hash; for a QR image it decodes the code with jsQR.
4. The client sends a request to the Express API (except for the password check, which contacts Have I Been Pwned directly).
5. The API validates the input again and checks the cache. If a fresh report exists, it is returned immediately.
6. The API queries the relevant security services in parallel. A failing service is marked "unavailable" and the others continue.
7. The threat score engine converts the findings into points, caps the total at 100 and assigns a verdict.
8. Gemini AI writes a short explanation of the findings; if it is unavailable, a rule-based explanation is used.
9. The JSON report is returned to the client, stored in the browser session, and the public scan counter is increased.
10. The result page shows the animated gauge, verdict badge, explanation, score breakdown, per-source findings and advice.
11. The user can download a PDF report, check the result again if VirusTotal is still analysing, or start another scan.

This methodology allows different security sources to work together in a single application while keeping the system fast, private and easy to understand.

CHAPTER 4 IMPLEMENTATION DETAILS

4.1 IMPLEMENTATION DETAILS

The CyberShield project is implemented as a web-based application with a React client and a Node.js API. The client is built with **React 19, Vite, Tailwind CSS v4 and Framer Motion**, and the API with **Node.js 20 and Express 5**. Cloud Firestore is used for the public scan counter.

The application is divided into different modules so that each function can be managed separately. The main modules implemented are:

- Link Scanner
- File Scanner
- SMS / Email Scam Checker
- QR Code Scanner
- IP Address Checker
- Password Leak Checker
- Threat Score Engine
- Result Page and PDF Report
- Awareness Page and Quiz
- Security middleware (CORS, validation, cache, rate limit)

Each module performs a specific function and works together with the others to provide the complete system.

4.2 LINK SCANNER IMPLEMENTATION

The Link Scanner accepts a URL from the user. The client checks that it is a valid http or https address and the server normalises it (adding https:// when no scheme is typed).

The server then runs four checks in parallel: Google Safe Browsing (threat types MALWARE, SOCIAL_ENGINEERING, UNWANTED_SOFTWARE and POTENTIALLY_HARMFUL_APPLICATION), VirusTotal (lookup by the base64url identifier of the URL, with submission and polling if the URL is unknown), RDAP domain age, and local heuristics such as HTTP instead of HTTPS, suspicious keywords (kyc, verify, login, otp, bank names), raw IP hosts, excessive subdomains, the "@" trick and punycode look-alike domains.

The findings are scored, explained by Gemini and returned with the full per-engine results so the result page can show an engine grid.

4.3 FILE SCANNER IMPLEMENTATION

The File Scanner uses a hash-first approach. The browser computes the SHA-256 hash of the selected file with the Web Crypto API and sends only the hash, name and size to the server. The server looks up the hash on VirusTotal; if the file is known, the complete report with detection count, malware family names and file type is returned instantly.

If the file is unknown, the server replies that an upload is needed, the client sends the file (up to 32 MB) as multipart data, and the server verifies that the uploaded bytes match the hash, uploads the file to VirusTotal and polls the analysis. Files are held only in memory and are never written to disk.

4.4 SMS / EMAIL SCAM CHECKER IMPLEMENTATION

The message checker sends the text to Gemini with a system prompt that demands strict JSON output: verdict, risk score, scam type, red-flag phrases with reasons, explanation and advice. At the same time a rule engine with sixteen patterns for Indian scams (KYC, OTP, account block threats, urgency, lottery, fake jobs, advance fees, QR "receive money", courier, loan apps and digital arrest, in English and Hinglish) analyses the same text.

All links found in the message are extracted with a regular expression and run through the Link Scanner. The result page highlights every red-flag phrase in the original message.

4.5 QR CODE SCANNER IMPLEMENTATION

The QR scanner offers three inputs: the device camera (getUserMedia with jsQR decoding of video frames), an uploaded image, or pasted QR text. The decoded content is classified as a URL, a UPI payment request, a Wi-Fi network, a phone number or plain text.

A URL is sent to the Link Scanner. A UPI request (upi://pay, gpay://, paytmmp://, phonepe://) is parsed for payee ID, name, amount and merchant code and scored locally, with the warning that nobody needs to scan a QR code to receive money.

4.6 PASSWORD LEAK CHECKER IMPLEMENTATION

The password checker runs entirely in the browser. The password is hashed with SHA-1, only the first five characters of the hash are sent to the Have I Been Pwned range API with the Add-Padding header, and the returned suffixes are compared locally to find the breach count. A local strength estimator calculates entropy, crack time, warnings and suggestions. The password is never sent to the project server and is cleared from the form on submit.

4.7 IP ADDRESS CHECKER IMPLEMENTATION

The IP checker validates IPv4 and IPv6 addresses with Node’s built-in parser, treats private and reserved ranges as non-public, and queries AbuseIPDB (abuse confidence score, reports, ISP, usage type, Tor flag) together with a free geolocation service for country, city, ASN, proxy and hosting flags.

4.8 THREAT SCORE ENGINE IMPLEMENTATION

All weights live in one configuration object so they are easy to explain and tune. For links: Safe Browsing match +50, each VirusTotal engine +10 (maximum +40), domain younger

than 30 days +15, HTTP +5, suspicious keywords +10, raw IP host +10, excessive subdomains +5 and "@" or punycode +10. Files score 8 points per detecting engine; IP addresses use the AbuseIPDB confidence score; messages use the Gemini risk score (or the rule score) plus half of the worst embedded link score. The total is capped at 100 and mapped to Safe, Suspicious or Dangerous. When a source cannot be checked the report lists it and the client shows a "Partial result" banner; when nothing can be scored the verdict is "Not scored" instead of a false Safe.

4.9 RESULT PAGE AND PDF REPORT IMPLEMENTATION

The result page shows a semicircular speedometer gauge whose needle animates from 0 to the score, a verdict badge, the explanation, a "Why this score?" breakdown, one card per source and the antivirus engine grid that lights up engine by engine. A Dangerous verdict makes the screen edges glow red. The jsPDF library generates a PDF report with the score box, summary, explanation and tables of the breakdown and findings.

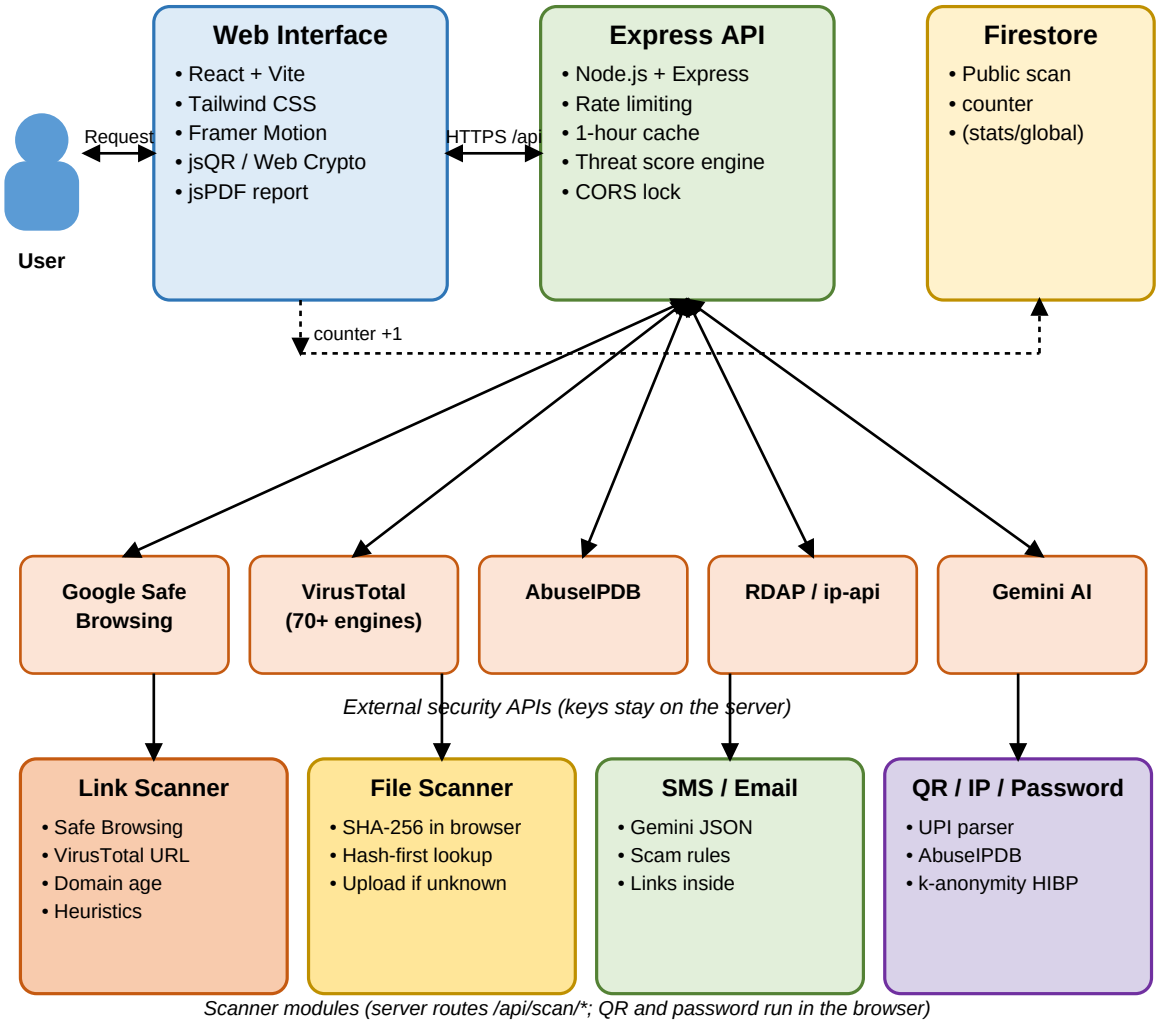
4.10 SECURITY AND DEPLOYMENT

The API restricts CORS to the configured client origin, limits JSON bodies to 64 kB and uploads to 32 MB, validates every input, caches reports for one hour and allows 20 scans per 10 minutes per IP. API keys are read from environment variables and never reach the browser. The client is deployed on GitHub Pages through a GitHub Actions workflow and the API on Render; Firestore security rules allow only a +1 increment of the public counter.

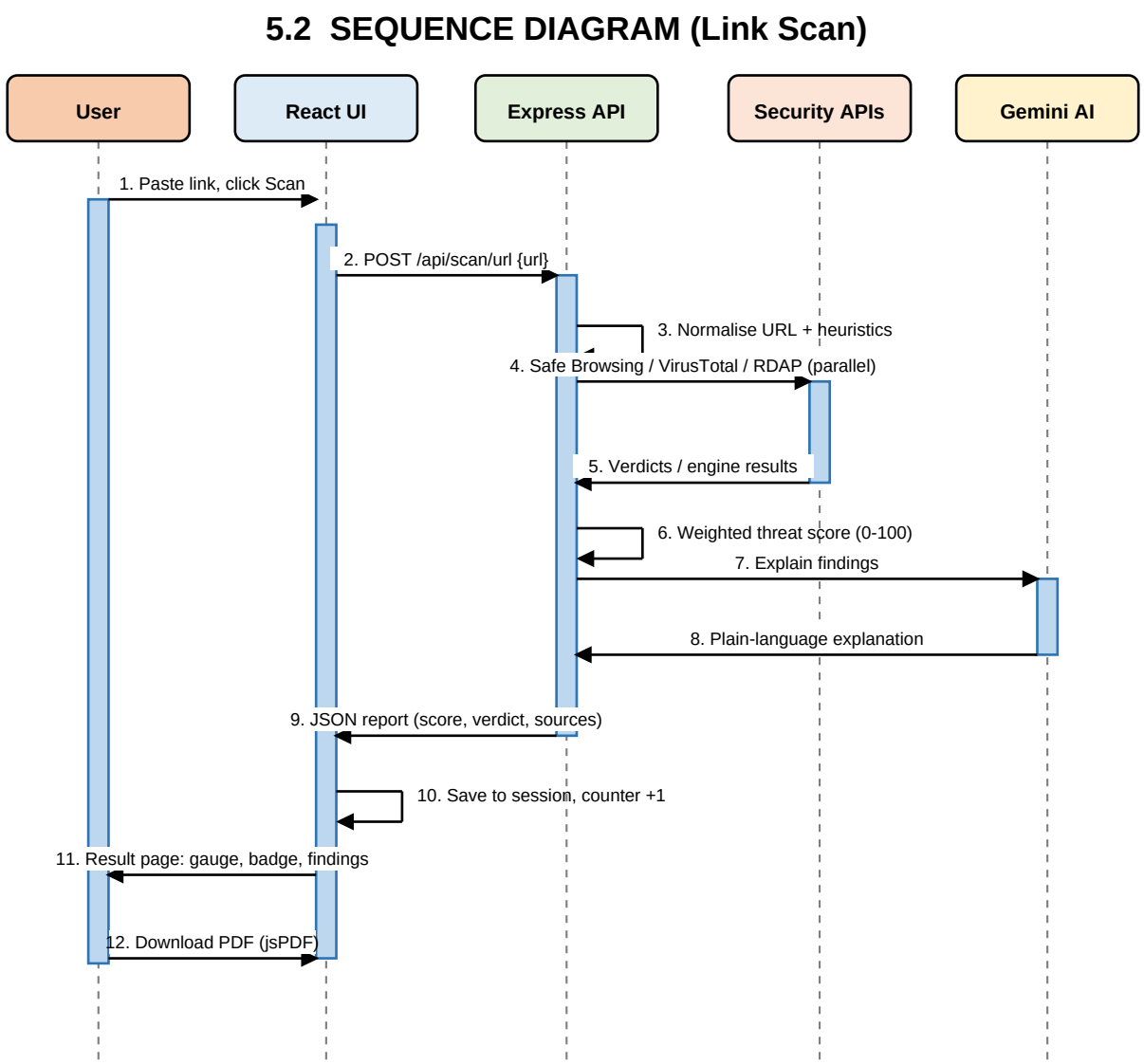
CHAPTER 5 SYSTEM DESIGN

5.1 Architecture Diagrams

5.1 ARCHITECTURE DIAGRAM

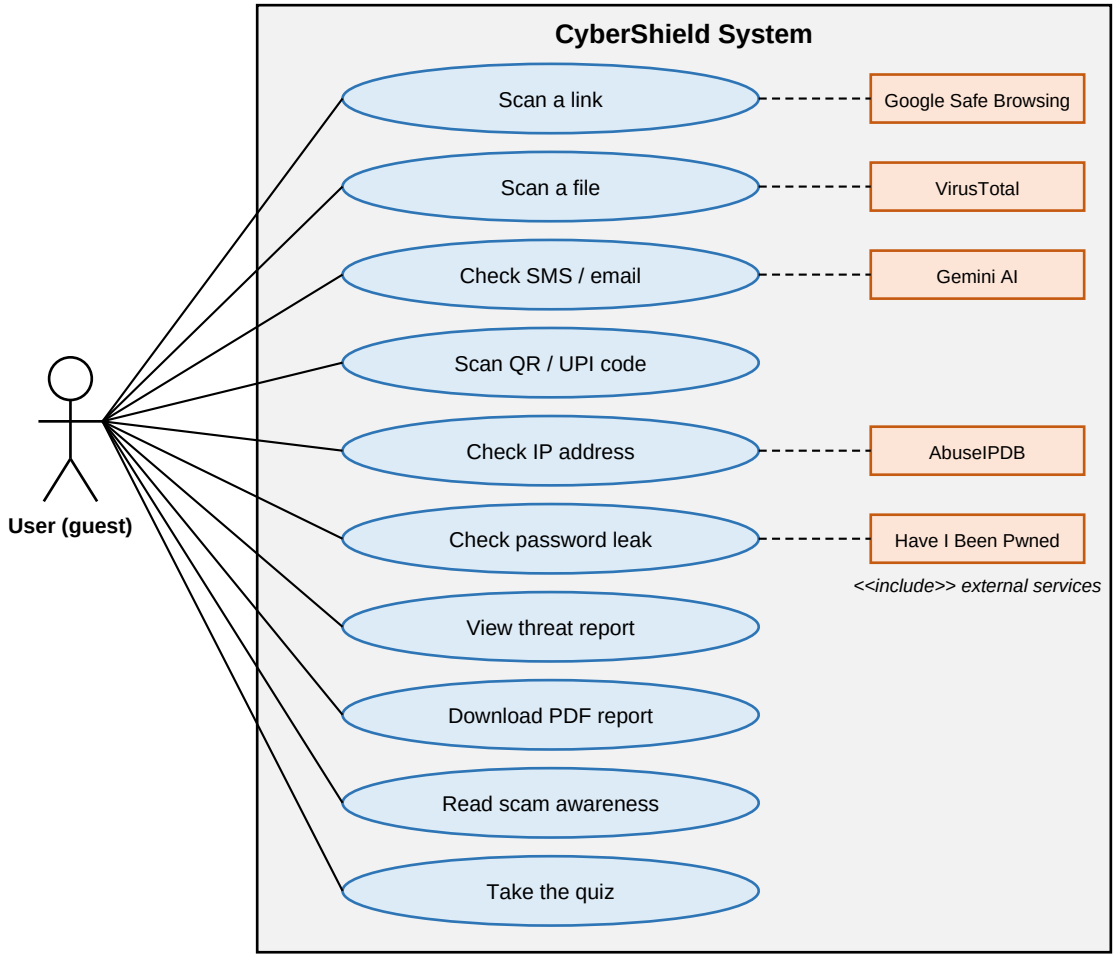


5.2 Sequence Diagrams:

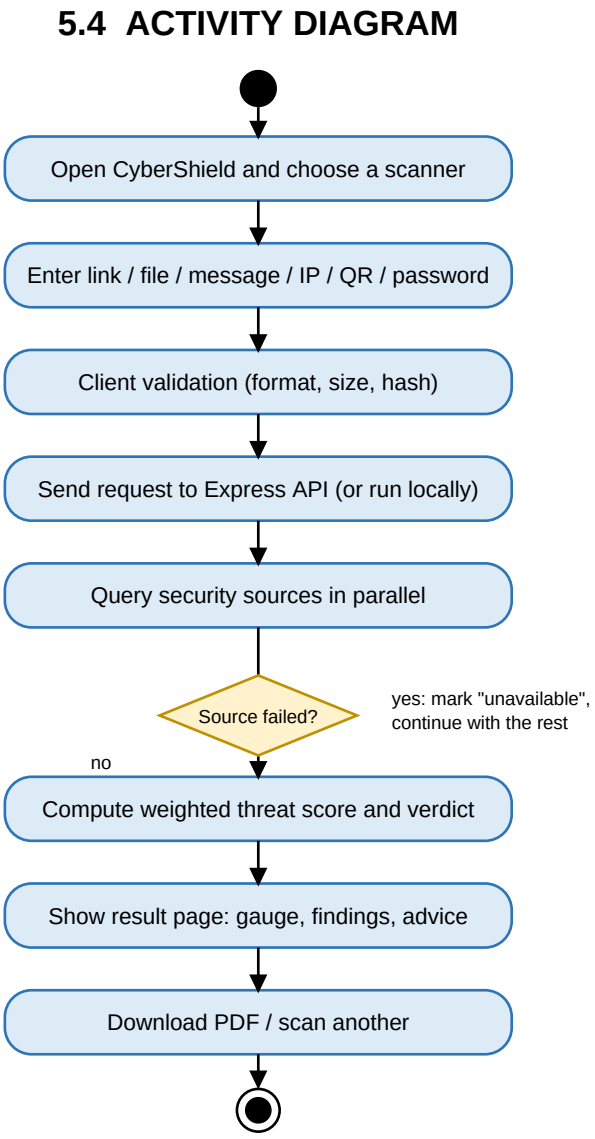


5.3 Use Case Diagrams:

5.3 USE CASE DIAGRAM



5.4 Activity Diagrams:



CHAPTER 6 EXPERIMENTAL SETUP & RESULT

6.1 EXPERIMENTAL SETUP

The CyberShield project was developed and tested as a web-based threat detection application. The system was tested locally with real API keys and later on the deployed site to check whether all the main features work correctly.

The testing was performed by using different inputs for every scanner and checking the output generated by the system. The main modules tested were the Link Scanner, File Scanner, SMS/Email Scam Checker, QR Code Scanner, IP Address Checker, Password Leak Checker, the threat score engine, the result page, the PDF report, the awareness quiz and the API security features.

The main purpose of the experimental setup was to verify that the system performs the required security checks correctly, handles failures gracefully and provides the expected results to the user.

6.2 HARDWARE SETUP

The following hardware setup was used for developing and testing the CyberShield project:

- Computer/Laptop: Personal Computer
- Processor: Apple M-series / Intel or AMD processor
- RAM: 8 GB or above
- Storage: 256 GB or above
- Input Devices: Keyboard, Mouse and Webcam (for QR scanning)
- Display: Standard Laptop/Monitor
- Internet Connection: Required for security API queries

The project does not require high-end hardware and can run on a normal computer, laptop or mobile phone browser.

6.3 SOFTWARE SETUP

The following software and technologies were used for developing the CyberShield project:

- Operating System: macOS / Windows / Linux
- Programming Language: JavaScript (ES modules)
- Frontend: React 19, Vite, Tailwind CSS v4, Framer Motion, React Router
- Backend: Node.js 20, Express 5, Axios, Multer, express-rate-limit
- Libraries: jsQR (QR decoding), jsPDF (PDF reports), @google/genai (Gemini)
- Database: Cloud Firestore (public scan counter)
- Security APIs: Google Safe Browsing v4, VirusTotal v3, AbuseIPDB v2, RDAP, ip-api, Have I Been Pwned
- Code Editor: Visual Studio Code
- Browser: Google Chrome

- Testing: Node.js built-in test runner, oxlint
- Deployment: GitHub Pages (client), Render (API), GitHub Actions

These software tools were used to develop, execute, test, deploy and debug the CyberShield application.

6.4 EXECUTION PROCEDURE

The CyberShield system was executed and tested using the following procedure:

1. The Express API was started with the API keys configured in the server environment file, and the health endpoint confirmed that all four keys were loaded.
2. The React client was started with Vite and opened in the browser.
3. Google’s official Safe Browsing test pages were scanned with the Link Scanner to confirm phishing and malware detection.
4. Genuine websites such as github.com were scanned to confirm a Safe verdict with no false alarms.
5. Look-alike domains, raw IP hosts, the "@" trick and punycode domains were scanned to test the heuristics.
6. The EICAR test file hash was looked up to confirm detection, and a new random file was uploaded to confirm the submit-and-poll flow and the "Check again" button.
7. KYC, OTP, job, lottery, courier and digital-arrest scam messages in English and Hinglish were checked, along with genuine messages, to test both the AI and the rule engine.
8. UPI "refund" QR codes, shop QR codes, Wi-Fi and link QR codes were decoded from images and pasted text.
9. Public, abusive, private and IPv6 addresses were checked with the IP Address Checker.
10. Common and strong passwords were checked against Have I Been Pwned and the strength meter.
11. The API was tested with invalid inputs, oversized bodies, a foreign origin and more than 20 requests in 10 minutes to confirm validation, CORS and rate limiting.
12. Automated tests were run with "npm test", the client was built, and the deployed site on GitHub Pages was opened and a scan was performed.

The system was tested with both valid and invalid inputs to verify proper validation, graceful degradation and error handling.

6.5 RESULT

The experimental testing showed that the CyberShield application successfully performs its main threat detection functions.

Google’s phishing test page scored 95 (Dangerous) with Safe Browsing flagging it and 13 VirusTotal engines detecting it, while github.com scored 0 (Safe). The EICAR test file was detected by 66 of 75 antivirus engines through its hash alone. The KYC scam message scored 88 with five highlighted red flags, and the fake job message scored 98 with Gemini explaining the scam. A known abusive IP address scored 79 with a 69% abuse confidence, and the password "password123" was found in more than 22 lakh breaches while the password itself

never left the browser.

Invalid inputs were rejected with clear messages, an unreachable service produced a "Partial result" banner instead of a wrong Safe verdict, the rate limiter blocked the 21st scan, and the deployed site worked end to end.

Overall, the testing results showed that the CyberShield system works correctly according to its designed functionality and provides a reliable platform for checking suspicious links, files, messages, QR codes, IP addresses and passwords.

Test Result Summary

Module	Result
Link Scanner (Safe Browsing + VirusTotal)	PASS
Link Heuristics and Domain Age	PASS
File Scanner (hash lookup)	PASS
File Scanner (upload and Check again)	PASS
SMS / Email Scam Checker (AI + rules)	PASS
Embedded Link Scan in Messages	PASS
QR Code Decoding and UPI Parsing	PASS
IP Address Checker	PASS
Password Leak Checker (k-anonymity)	PASS
Threat Score Engine	PASS
Result Page and PDF Report	PASS
Input Validation, CORS and Rate Limiting	PASS
Awareness Quiz	PASS
Deployment (GitHub Pages + Render)	PASS

Overall Result: SUCCESSFUL

6.6 SCREENSHOTS

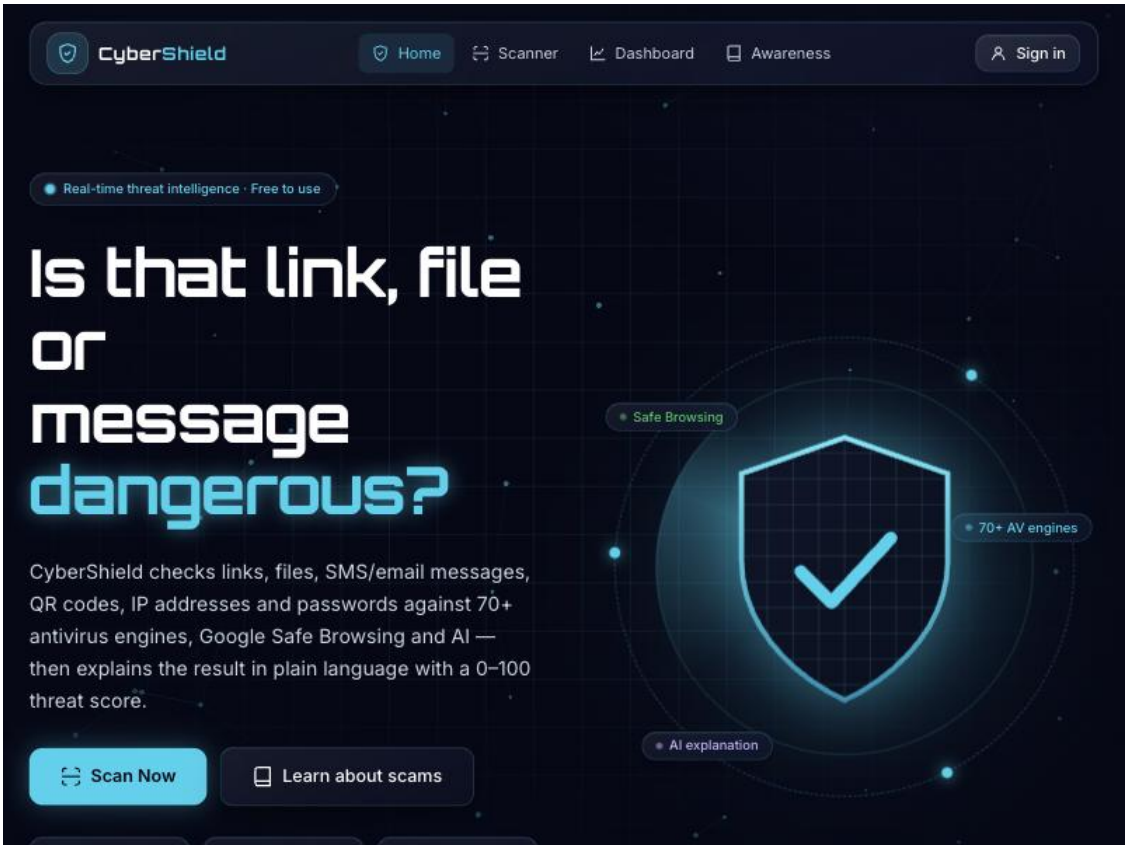


Fig 6.1 Home page with scanner cards and live scan counter

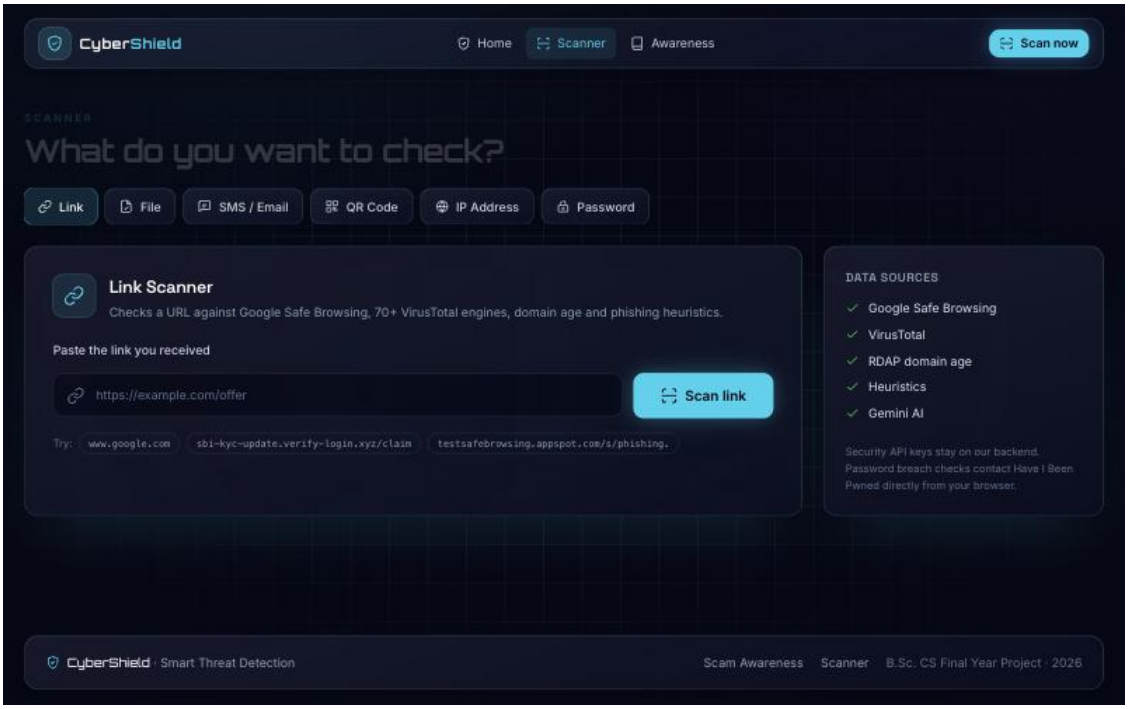


Fig 6.2 Scanner page with the six scanner tabs

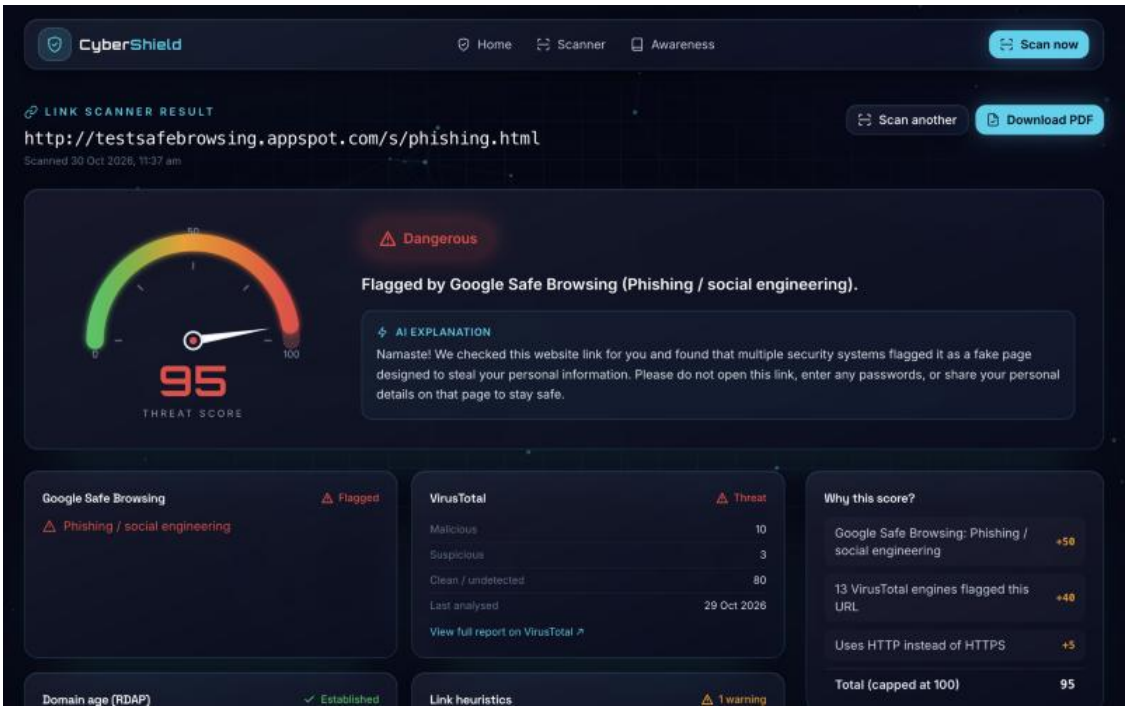


Fig 6.3 Result page: threat gauge, verdict and AI explanation

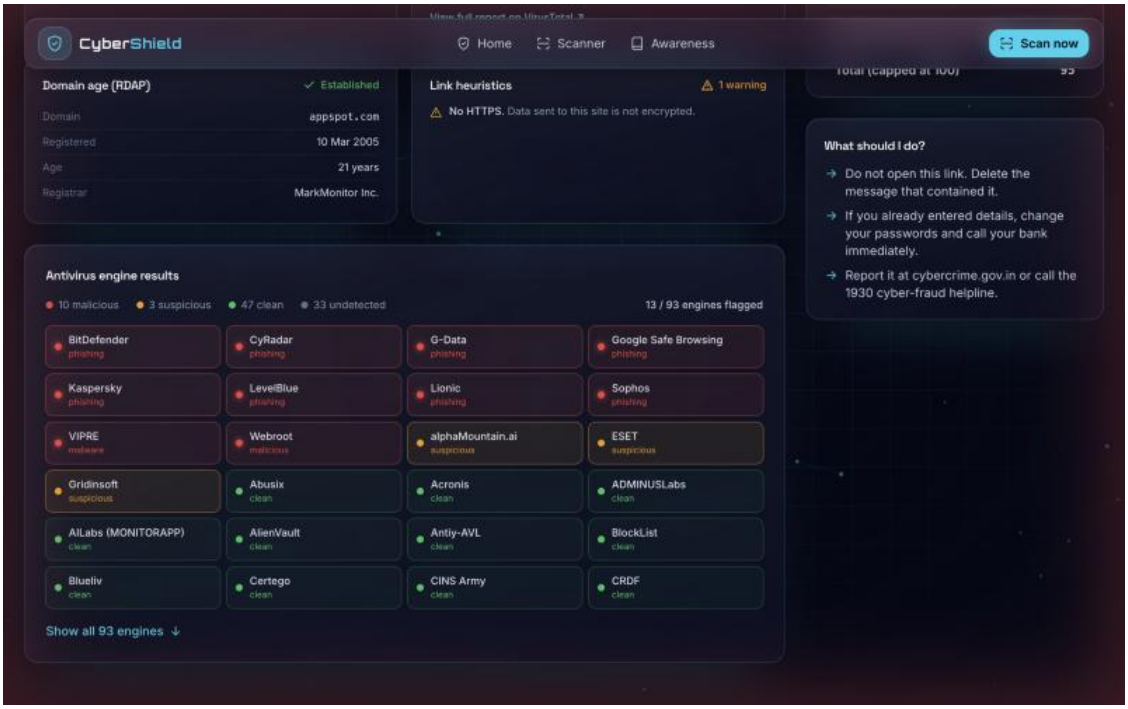


Fig 6.4 Antivirus engine grid and score breakdown

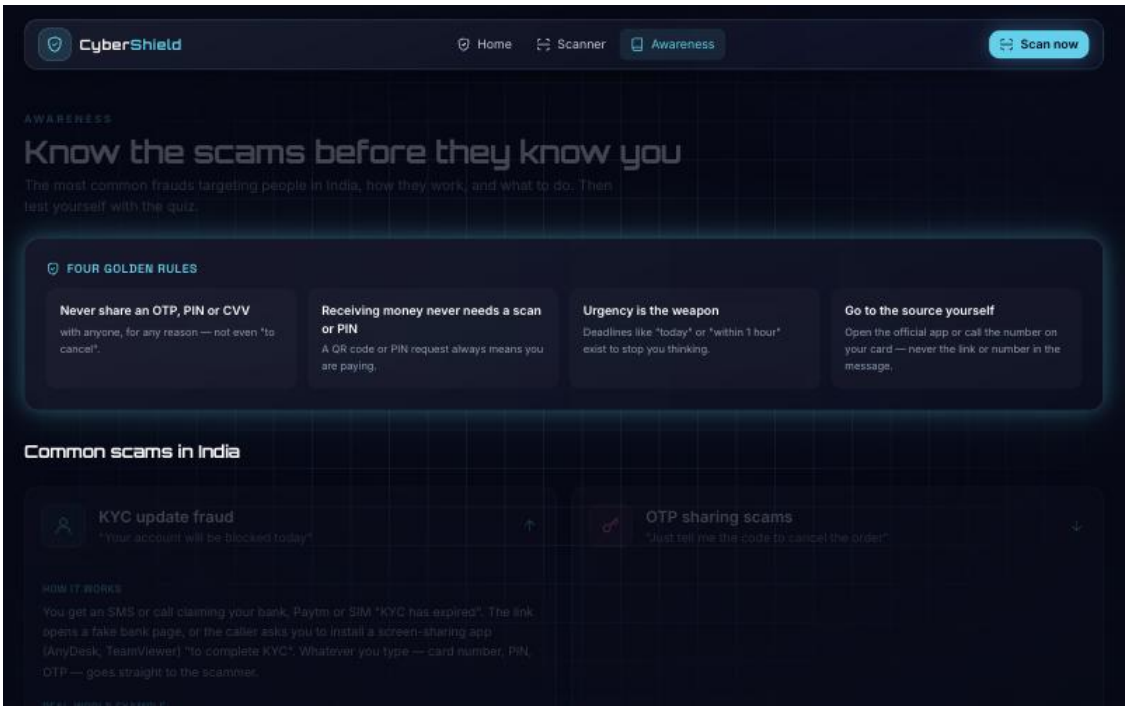


Fig 6.5 Scam awareness page

CHAPTER 7 ANALYSIS OF REPORT

7.1 ANALYSIS OF RESULT

The testing of the CyberShield system was performed to check whether the implemented features work as expected. The major modules such as the Link Scanner, File Scanner, SMS/Email Checker, QR Scanner, IP Checker, Password Checker, threat score engine, result page, PDF report, API security and deployment were tested.

The test results show that all the major modules of the system worked successfully. The system was able to detect known phishing and malware test pages, identify a test malware file by its hash, classify scam messages, decode QR codes, rate IP addresses and find leaked passwords.

Therefore, the overall result of the testing was successful.

7.2 ACCURACY ANALYSIS

The accuracy of the CyberShield system was checked by providing known-dangerous and known-safe inputs during testing.

Google’s official Safe Browsing test pages were correctly flagged as Dangerous and popular genuine websites were rated Safe. The EICAR test file was detected correctly. Scam messages received high scores with correct scam types, while a genuine bank OTP reminder that says "never share your OTP" was correctly rated Safe after the OTP rule was refined. A UPI refund QR was rated Dangerous while a registered shop QR was rated Safe.

Because the scores come from several independent sources, a single wrong signal cannot by itself produce a Dangerous verdict, which reduces false alarms. The testing results indicate that the system provides accurate responses for the implemented functionalities.

7.3 RELIABILITY AND CONSISTENCY

The CyberShield system was tested multiple times using different inputs to check its reliability and consistency.

The system consistently performed important operations such as link, file, message and IP scanning. Error conditions such as invalid URLs, malformed IP addresses, files larger than 32 MB, mismatched file hashes, blank messages, a stopped API and exhausted AI quota were all handled properly: the user received a clear message, and a missing source was reported as "unavailable" or "Not scored" rather than as a false Safe.

The results show that the system provides consistent output for the tested operations.

7.4 REAL TIME PERFORMANCE ANALYSIS

The CyberShield application was tested in a local environment and on the deployed GitHub Pages site. A link scan that is already known to VirusTotal completes in about two to three seconds; a brand-new link takes longer because VirusTotal must analyse it, and the result

page offers a "Check again" button. Cached scans return in under one second. Message analysis with Gemini completes in about three to five seconds, and the AI explanation step is capped so that a slow model can never delay a scan by more than a few seconds.

Since the VirusTotal free tier allows about four requests per minute and the Gemini free tier has daily limits, the one-hour cache, model fallback chain and rule-based fallback keep the application usable even when a service is busy.

7.5 RESULT INTERPRETATION

Based on the experimental testing and analysis, the CyberShield system successfully achieved its main objectives.

The system provides link, file, message, QR, IP and password checking, a weighted threat score with a clear verdict, plain-language explanations, a downloadable PDF report and scam awareness content, while keeping API keys on the server and passwords in the browser.

All the tested modules produced the expected results, and the overall test result was SUCCESSFUL. The project demonstrates the practical implementation of threat intelligence and privacy-preserving techniques in a modern web application.

CHAPTER 8 FUTURE ENHANCEMENTS

Although the current CyberShield system provides the main features required for checking suspicious links, files, messages, QR codes, IP addresses and passwords, the system can be improved further in the future.

The following enhancements can be added:

1. User Accounts and Scan History

Optional sign-in with a personal dashboard of past scans, charts and trends.

2. Browser Extension

A Chrome extension that checks links automatically before the user clicks them.

3. Mobile Application

An Android application that can scan SMS messages and QR codes directly on the phone.

4. Additional Threat Sources

More reputation feeds such as PhishTank, URLhaus and OpenPhish, and WHOIS data for more TLDs.

5. Screenshot and Visual Phishing Detection

Rendering a page in a sandbox and comparing it with known brand pages to detect visual phishing.

6. Regional Language Support

Scam detection and explanations in Hindi, Marathi and other Indian languages.

7. Reporting to Authorities

A one-click option to report a confirmed scam to the national cybercrime portal.

8. Offline Rule Updates

Downloadable updates to the scam pattern rules so the message checker improves without a new release.

These enhancements can make CyberShield more secure, reliable and useful for users while providing better protection against online fraud.

CHAPTER 9 PROGRAM CODE

9.1 Implementation Code

The **CyberShield** project is developed using JavaScript on both sides. The main API file is `server/index.js`, which configures Express, CORS, rate limiting and the scan routes. The scanning logic lives in `server/services/` and the scoring logic in `server/utils/threatScore.js`. The React client lives in `client/src/`.

9.1.1 Express Application Setup

The Express application is created with CORS restricted to the configured client origin, a small JSON body limit and a rate limiter on all scan routes.

#CODE

```
import express from 'express';
import cors from 'cors';
import scanRouter from './routes/scan.js';
import { scanLimiter } from './middleware/rateLimiter.js';

const app = express();
app.set('trust proxy', 1);
const allowedOrigins = allowedClientOrigins(process.env.CLIENT_ORIGIN);
app.use(cors({
  origin(origin, cb) {
    if (!origin || allowedOrigins.includes(origin)) return cb(null, true);
    return cb(httpError(403, 'forbidden_origin',
      'This browser origin is not allowed.'));
  },
  methods: ['GET', 'POST'],
}));
app.use(express.json({ limit: '64kb' }));
app.use('/api/scan', scanLimiter, scanRouter);
app.listen(PORT);
```

This code creates the API server and makes sure that only the CyberShield website can call it.

9.1.2 Rate Limiting and Caching

The rate limiter allows 20 scans per 10 minutes per IP address, and a tiny in-memory cache stores reports for one hour to protect the VirusTotal quota.

#CODE

```
export const scanLimiter = rateLimit({
  windowMs: 10 * 60 * 1000,
  limit: 20,
  message: { error: 'rate_limited',
             message: 'Too many scans. Please wait a bit.' },
});

const store = new Map();
export function cacheGet(key) {
  const hit = store.get(key);
  if (!hit) return undefined;
  if (Date.now() > hit.expires) { store.delete(key); return undefined; }
  return hit.value;
```

```
}
export function cacheSet(key, value, ttlMs = 60 * 60 * 1000) {
  store.set(key, { value, expires: Date.now() + ttlMs });
  return value;
}
```

9.1.3 Threat Score Engine

All weights live in one configuration object. The link scorer adds points for each signal, caps the total at 100 and returns a breakdown that the result page displays.

#CODE

```
export const WEIGHTS = {
  url: { SAFE_BROWSING_MATCH: 50, VT_PER_ENGINE: 10, VT_MAX: 40,
         YOUNG_DOMAIN: 15, YOUNG_DOMAIN_DAYS: 30, NO_HTTPS: 5,
         SUSPICIOUS_KEYWORDS: 10, IP_ADDRESS_URL: 10,
         EXCESSIVE_SUBDOMAINS: 5, AT_SYMBOL_OR_PUNYCODE: 10 },
};
export const THRESHOLDS = { SAFE_MAX: 30, SUSPICIOUS_MAX: 60 };

export function verdictFromScore(score) {
  if (score <= THRESHOLDS.SAFE_MAX) return 'safe';
  if (score <= THRESHOLDS.SUSPICIOUS_MAX) return 'suspicious';
  return 'dangerous';
}

export function scoreUrl({ safeBrowsing, virusTotal, domainAge, heuristics })
{
  const W = WEIGHTS.url; const breakdown = []; let score = 0;
  if (safeBrowsing?.available && safeBrowsing.matched) {
    score += W.SAFE_BROWSING_MATCH;
    breakdown.push({ label: 'Google Safe Browsing match', points: W.SAFE_BROWS
ING_MATCH });
  }
  if (virusTotal?.available && virusTotal.stats) {
    const flagged = virusTotal.stats.malicious + virusTotal.stats.suspicious;
    if (flagged > 0) {
      const pts = Math.min(W.VT_MAX, flagged * W.VT_PER_ENGINE);
      score += pts;
      breakdown.push({ label: `${flagged} VirusTotal engines flagged this URL`
, points: pts });
    }
  }
  if (domainAge?.available && domainAge.ageDays < W.YOUNG_DOMAIN_DAYS) {
    score += W.YOUNG_DOMAIN;
    breakdown.push({ label: `Domain registered only ${domainAge.ageDays} days
ago`, points: W.YOUNG_DOMAIN });
  }
  if (!heuristics.isHttps) { score += W.NO_HTTPS; breakdown.push({ label: 'Use
s HTTP instead of HTTPS', points: W.NO_HTTPS }); }
  const finalScore = Math.min(100, score);
  return { score: finalScore, verdict: verdictFromScore(finalScore), breakdown
 };
}
```

This function converts the findings of every source into one score between 0 and 100 and a verdict.

9.1.4 Link Scanner Orchestrator

The orchestrator runs all sources in parallel. A failing source returns an "unavailable" object instead of throwing, so one slow or broken API never blocks the others.

#CODE

```
const settled = (p) => p.then((v) => v).catch(() => ({ available: false, reaso
n: 'error' }));

export async function scanUrl(rawUrl, { skipCache = false, quick = false } = {
}) {
  const u = normalizeUrl(rawUrl);
  if (!u) throw httpError(400, 'invalid_url', 'Please enter a valid http:// or
 https:// link.');
  const url = u.href;
  const cached = skipCache ? null : cacheGet(`url:${url}`);
  if (cached) return { ...cached, cached: true };

  const heuristics = analyzeUrlHeuristics(u);
  const [safeBrowsing, virusTotal, domainAge] = await Promise.all([
    settled(checkSafeBrowsing(url)),
    settled(scanUrlWithVirusTotal(url, { quick })),
    settled(getDomainAge(getRegistrableDomain(u.hostname))),
  ]);
  const { score, verdict, breakdown } = scoreUrl({ safeBrowsing, virusTotal, d
omainAge, heuristics });
  const ai = await explainFindings({ scanType: 'url', input: url, verdict, sco
re, breakdown });
  const report = { scanType: 'url', input: url, threatScore: score, verdict, e
xplanation: ai.text,
                   breakdown, sources: { safeBrowsing, virusTotal, domainAge,
heuristics } };
  return cacheSet(`url:${url}`, report);
}
```

9.1.5 Google Safe Browsing Lookup

The Safe Browsing v4 API is queried with the four threat types used by the project.

#CODE

```
const THREAT_TYPES = ['MALWARE', 'SOCIAL_ENGINEERING', 'UNWANTED_SOFTWARE', 'P
OTENTIALLY_HARMFUL_APPLICATION'];

export async function checkSafeBrowsing(url) {
  const { data } = await axios.post(
    `https://safebrowsing.googleapis.com/v4/threatMatches:find?key=${process.e
nv.SAFE_BROWSING_API_KEY}`,
    { client: { clientId: 'cybershield', clientVersion: '1.0.0' },
      threatInfo: { threatTypes: THREAT_TYPES, platformTypes: ['ANY_PLATFORM']
,
                    threatEntryTypes: ['URL'], threatEntries: [{ url }] } },
    { timeout: 8000 });
  const matches = data?.matches || [];
  return { available: true, matched: matches.length > 0, threats: matches.map(
(m) => m.threatType) };
}
```

9.1.6 VirusTotal URL and File Lookup

VirusTotal identifies a URL by its base64url encoding without padding. A file is identified by its SHA-256 hash; unknown files are uploaded and the analysis is polled.

#CODE

```
export const vtUrlId = (url) =>
  Buffer.from(url).toString('base64').replace(/\+/g, '-').replace(/\//g, '_').
replace(/=+$/, '');

export async function lookupFileHash(sha256) {
  try {
    const { data } = await client().get(`/files/${sha256}`);
    const attrs = data.data.attributes;
    return fromAttributes(attrs, { found: true,
      fileType: attrs.type_description,
      popularThreat: attrs.popular_threat_classification?.suggested_threat_lab
el || null });
  } catch (err) {
    if (err.response?.status === 404) return { available: true, status: 'not_f
ound', found: false };
    return handleError(err, 'get-file');
  }
}
```

9.1.7 Hash-First File Scan

The server verifies that uploaded bytes really match the hash sent by the browser before contacting VirusTotal, and never writes the file to disk.

#CODE

```
export async function scanFile({ sha256, fileName, size, mimeType, buffer }) {
  if (!/^[a-f0-9]{64}$/i.test(sha256 || '')) throw httpError(400, 'invalid_has
h', 'A valid SHA-256 hash is required.');
  if (buffer && createHash('sha256').update(buffer).digest('hex') !== sha256.t
oLowerCase()) {
    throw httpError(400, 'hash_mismatch', 'The uploaded file does not match it
s SHA-256 hash.');
  }
  let vt = await lookupFileHash(sha256);
  if (vt.available && vt.status === 'not_found') {
    if (!buffer) return { needsUpload: true, sha256 };
    vt = await uploadFileToVirusTotal(buffer, fileName);
    buffer = null; // drop the bytes immediately
  }
  const { score, verdict, breakdown } = scoreFile({ virusTotal: vt });
  return { scanType: 'file', sha256, threatScore: score, verdict, breakdown, s
ources: { virusTotal: vt } };
}
```

9.1.8 Browser-Side SHA-256 and SHA-1 Hashing

The Web Crypto API computes the file fingerprint and the password hash inside the browser.

#CODE

```
const toHex = (buf) => [...new Uint8Array(buf)].map((b) => b.toString(16).padS
tart(2, '0')).join('');

export async function sha256Hex(file) {
  const digest = await crypto.subtle.digest('SHA-256', await file.arrayBuffer(
```

```
));
  return toHex(digest);
}

export async function sha1HexUpper(text) {
  const digest = await crypto.subtle.digest('SHA-1', new TextEncoder().encode(
text));
  return toHex(digest).toUpperCase();
}
```

9.1.9 Password Check with k-Anonymity

Only the first five characters of the SHA-1 hash are sent to Have I Been Pwned. The suffix comparison happens locally, so the password never leaves the browser.

#CODE

```
export async function checkPwned(password) {
  const hash = await sha1HexUpper(password);
  const prefix = hash.slice(0, 5);
  const suffix = hash.slice(5);
  const res = await fetch(`https://api.pwnedpasswords.com/range/${prefix}`, {
headers: { 'Add-Padding': 'true' } });
  let breachCount = 0;
  for (const line of (await res.text()).split('\n')) {
    const [sfx, count] = line.trim().split(':');
    if (sfx === suffix) breachCount = Number(count) || 0;
  }
  return { available: true, prefix, breachCount };
}
```

9.1.10 Gemini Scam Analysis with Strict JSON

The message checker asks Gemini for a strict JSON answer and validates every field before using it.

#CODE

```
const MESSAGE_SYSTEM_PROMPT = `You are a scam-detection expert for SMS and ema
il messages, especially Indian scams
(KYC fraud, OTP theft, fake job offers, electricity-bill threats, lottery, UPI
 "receive money", courier, loan apps).
Respond with STRICT JSON ONLY: {"verdict":"safe|suspicious|scam","riskScore":0
-100,"scamType":"string",
"redFlags":[{"phrase":"exact phrase","reason":"why"}],"explanation":"2-3 sente
nces","advice":"1-2 sentences"}`;

export async function analyzeMessage(text) {
  const res = await generate({ contents: text,
    config: { systemInstruction: MESSAGE_SYSTEM_PROMPT, responseMimeType: 'app
lication/json', temperature: 0.2 } });
  const json = JSON.parse(res.text);
  return { available: true,
    verdict: ['safe', 'suspicious', 'scam'].includes(json.verdict) ? json.verd
ict : 'suspicious',
    riskScore: Math.max(0, Math.min(100, Number(json.riskScore) || 0)),
    scamType: String(json.scamType || 'none'),
    redFlags: (json.redFlags || []).slice(0, 10), explanation: String(json.exp
lanation || ''), advice: String(json.advice || '') };
}
```

9.1.11 Scam Rule Engine

A set of regular expressions recognises common Indian scam patterns and works as a fallback when the AI is unavailable.

#CODE

```
const PATTERNS = [
  { re: /\b(kyc|know your customer)\b/i, pts: 20, type: 'KYC fraud', reason: '
Banks never ask you to "update KYC" through a link.' },
  { re: /\b(won|winner|lottery|lucky draw|prize|jackpot)\b/i, pts: 20, type: '
Lottery / prize scam', reason: 'You cannot win a lottery you never entered.' }
,
  { re: /\b(scan|qr code)\b.{0,30}\b(receive|get|collect)\b/i, pts: 25, type:
'QR "receive money" scam', reason: 'You NEVER need to scan a QR code to receiv
e money.' },
  { re: /\b(band|block)\s+(ho|kar)\s*(jayega|jayegi)\b/i, pts: 20, type: 'Acco
unt-block threat', reason: '"Band ho jayega" is a pressure tactic.' },
];
export function analyzeMessageHeuristics(text) {
  const redFlags = []; let score = 0;
  for (const p of PATTERNS) {
    const m = text.match(p.re);
    if (m) { score += p.pts; redFlags.push({ phrase: m[0].trim(), reason: p.re
ason }); }
  }
  const riskScore = Math.min(100, score);
  return { available: true, source: 'rules', riskScore, redFlags,
           verdict: riskScore <= 30 ? 'safe' : riskScore <= 60 ? 'suspicious'
: 'scam' };
}
```

9.1.12 QR Code Content Parsing

The decoded QR string is classified, and UPI payment requests are parsed for the payee, amount and merchant code.

#CODE

```
export function parseQrContent(raw) {
  const text = String(raw || '').trim();
  if (/^(upi|gpay|paytmmp|phonepe):\/\//i.test(text)) {
    const p = new URLSearchParams(text.slice(text.indexOf('?') + 1));
    return { kind: 'upi', raw: text, upi: { payeeAddress: p.get('pa') || '', p
ayeeName: p.get('pn') || '',
             amount: p.get('am') ? Number(p.get('am')) : null, merchantCode: p
.get('mc') || '' } };
  }
  if (/^https?:\/\//i.test(text) || /^www\./i.test(text)) return { kind: 'url'
, raw: text, url: text };
  if (/^WIFI:/i.test(text)) return { kind: 'wifi', raw: text };
  if (/^tel:/i.test(text)) return { kind: 'tel', raw: text, phone: text.slice(
4) };
  return { kind: 'text', raw: text };
}
```

9.1.13 Scan Route and Client Request

The scan route receives the link, and the React client calls it through a small fetch wrapper, then navigates to the result page.

#CODE

```
// server/routes/scan.js
router.post('/url', async (req, res) => {
  const { url } = req.body || {};
  if (typeof url !== 'string' || !url.trim()) throw httpError(400, 'missing_ur
l', 'Please provide a link to scan.');
  res.json(await scanUrl(url, { skipCache: req.query.fresh === '1' }));
});

// client/src/pages/Scanner.jsx
const runScan = async (fn) => {
  setScanning(true);
  try {
    const [result] = await Promise.all([fn(abortRef.current.signal), wait(MIN_
SCAN_MS)]);
    saveLastResult(result);
    bumpGlobalCounter();
    navigate('/result', { state: { result } });
  } catch (err) {
    setError({ type, value: err });
  } finally { setScanning(false); }
};
```

9.1.14 Firestore Security Rules

The only document anyone may write is the public counter, and only by adding exactly one.

#CODE

```
service cloud.firestore {
  match /databases/{database}/documents {
    match /stats/global {
      allow read: if true;
      allow update: if request.resource.data.keys().hasOnly(['totalScans'])
                    && request.resource.data.totalScans == resource.data.total
Scans + 1;
    }
    match /{document=**} { allow read, write: if false; }
  }
}
```

9.2 Running the Application

Both servers are started from the project folder with a single command, and the client is deployed by a GitHub Actions workflow.

#COMMANDS

```
npm run install:all      # install root, server and client dependencies
npm run dev              # starts the API on :5001 and the client on :5173
npm test                 # runs the server and client test suites
npm run build            # production build of the client (client/dist)
```

This starts the CyberShield API and website for local development.

CHAPTER 10 CONCLUSION

The CyberShield project was developed as a web-based smart threat detection application to give ordinary users a simple and reliable way to check whether a link, file, SMS or email message, QR code, IP address or password is dangerous. The system combines Google Safe Browsing, VirusTotal, RDAP, AbuseIPDB, Have I Been Pwned and Gemini AI behind one interface and presents a weighted threat score, a clear verdict and a plain-language explanation.

The project uses React, Vite, Tailwind CSS and Framer Motion for the client, Node.js and Express for the API, and Cloud Firestore for the public scan counter. Files are identified by a hash computed in the browser, passwords are checked with k-anonymity so they never leave the browser, API keys stay on the server, and the API is protected by CORS, validation, caching and rate limiting.

The testing of the major modules showed that the implemented features work successfully. Known phishing and malware test pages were flagged, a test malware file was detected by its hash, scam messages in English and Hinglish were classified with highlighted red flags, UPI "receive money" QR codes were warned about, abusive IP addresses were rated, leaked passwords were found, and the application was deployed to GitHub Pages and Render.

Overall, CyberShield demonstrates how professional threat intelligence services and privacy-preserving techniques can be combined into a single, user-friendly web application. The project helped in understanding practical concepts such as threat scoring, hashing, k-anonymity, REST API design, graceful degradation, AI integration and cloud deployment. It provides a useful foundation for developing more advanced security tools in the future.

REFERENCES

1. Google Safe Browsing API Documentation – Lookup API v4.
2. VirusTotal API v3 Documentation – URL and file analysis.
3. AbuseIPDB API Documentation – IP address reputation.
4. Have I Been Pwned – Pwned Passwords API and k-anonymity.
5. Google Gemini API Documentation – Generative AI with JSON output.
6. RDAP (Registration Data Access Protocol) – IETF RFC 9083.
7. React Documentation – User interface library.
8. Vite Documentation – Front-end build tool.
9. Tailwind CSS Documentation – Utility-first CSS framework.
10. Node.js and Express Documentation – Server-side JavaScript and web framework.
11. Firebase Cloud Firestore Documentation – Database and security rules.
12. jsQR and jsPDF Documentation – QR decoding and PDF generation libraries.
13. Indian Cyber Crime Coordination Centre – National Cyber Crime Reporting Portal (cybercrime.gov.in) and helpline 1930.